

约束求解简介

第二次大作业

Xidian-ICTT-YB/Mathematical_Logic_Course: 2024数理逻辑课程大作业（西安电子科技大学拔尖班大三年级）

Xidian-ICTT-YB / Mathematical_Logic_Course

Code

Issues

Pull requests

Actions

Projects

Wiki

Security

Insights

Settings

Search

Type / to search

Mathematical_Logic_Course

Public

Edit Pins

Watch 0

Fork 8

Starred 1

main

1 Branch

0 Tags

Go to file

Add file

Code

About

BinYu-Xidian-University

Merge pull request #12 from liptile/patch-1

dc1d4b2 · 10 months ago

65 Commits

第1组：对TLA+的介绍和展示	第一小组TLA+课上展示视频	11 months ago
第2组：Z3求解器的介绍与演示	Create 20241205-第二小组Z3求解器课上展示视频.mp4	10 months ago
第3组：cvc5的介绍与使用演示	Update README.md	10 months ago
第4组：ESBMC检测器的介绍与演示	展示视频上传	10 months ago
第5组：KLEE的介绍与演示	展示视频上传	10 months ago
第6组：LASVCSEQ工具的搭建和使用	第六组	10 months ago
第7组：论文检索方法介绍与演示	Add files via upload	10 months ago
.DS_Store	Create .DS_Store	11 months ago
README.md	Update README.md	10 months ago

README

About

2024数理逻辑课程大作业（西安电子科技大学拔尖班大三年级）

Readme

Activity

Custom properties

1 star

0 watching

8 forks

Report repository

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

- 约束问题
- 约束模型
 - SAT
 - SMT
 - 整数线性规划
 - CSP
- 约束求解
 - 完备方法
 - 不完备方法

生活中的约束问题



时间规划（一天/一个月的时间如何安排？）



通勤
（规避拥堵、可顺道买早餐？）



工作
（优先客户/上级/团队？）



点餐
（什么平台？菜系？门店？送餐时效？）



不动产购置
（宜居/便利/学位/医疗）



生意投资
（资源有限情况下，是否投/投什么/影响变量/回报要求）



职业发展
（意向行业/回报/区域/）



疫情防范
（物资储备/规避区域/应对策略）

工业中的约束问题



约束求解器是EDA(电子设计自动化)的逻辑设计、验证、测试、仿真、调优等环节的计算引擎。



信息安全领域是约束算法的典型应用领域，包括密码分析，算法安全等。



资源调度约束包括业务需求、运载能力、停靠能力、时间窗口、天气环境等，其约束算法在港口码头、运输规划、运力配载、生产排班等均有应用需求。

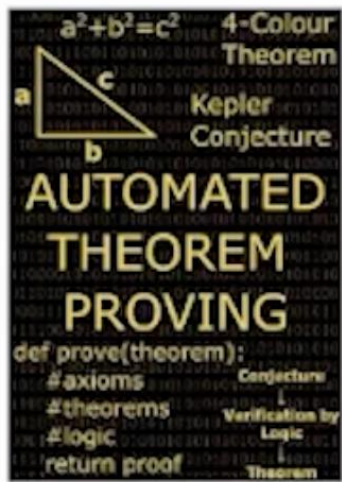


形式化验证是确保软件正确性的重要技术，尤其是对研发代价昂贵，安全攸关的软件，比如国防、航空航天、医疗等软件。约束求解器是软件验证不可或缺的引擎。

两种计算思维

符号主义（约束求解）

- 问题可以清晰描述
- 用户描述问题，计算机解决问题
- Typical：定理证明



连接主义（机器学习）

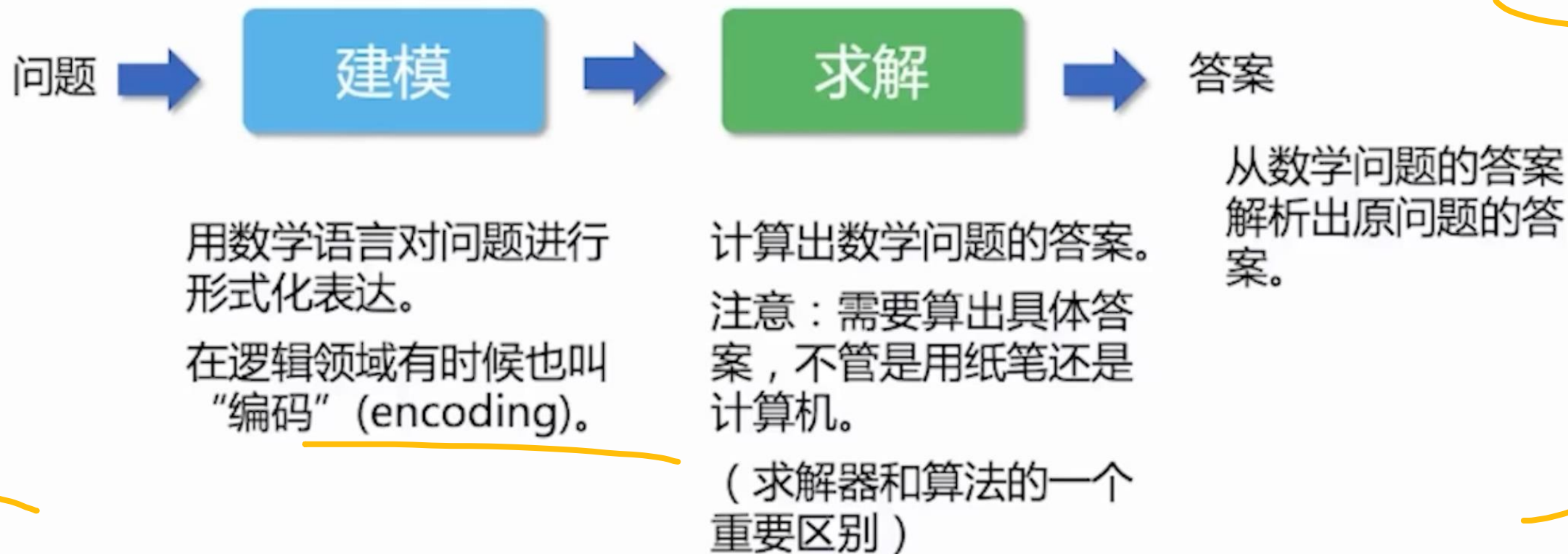
- 问题难以清晰描述
- 用户提供例子，计算机解决问题
- Typical：模式识别



神经符号系统（Neuro-Symbolic System）

是当代人工智能（AI）研究的一个核心方向，旨在结合神经网络的感知与符号系统的推理能力。

约束求解



约束数学语言

命题逻辑 (SAT) : 侧重逻辑约束

$$\varphi = (x_1 \vee \neg x_2) \wedge (x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_3 \vee x_4)$$

变量关系 (CSP) : 各种类型的约束

$$\psi = AllDifferent(x_i, x_j) \wedge |x_i - x_j| \neq |i - j|$$

数学规划 (MP) : 侧重数值约束

$$y^2 + 2xy < 100$$

$$x - y < 30$$

$$x \leq 60$$

^{SAT}
一阶逻辑 (SMT) : 逻辑+背景理论

$$\Phi = ((y^2 + 2xy < 100) \vee ((f(y) < 30)$$

$$\rightarrow \neg(x - y < 30) \wedge (x \leq 60))$$

* CSP全称是Constraint Satisfaction Problem, 约束满足问题, 广义上可以包括所有约束问题, 不过实际中, CSP往往聚焦于某些语法, 也即狭义的CSP。

问题分类

按目标：

判定问题：回答 Yes or No [e.g., SAT]

优化问题：满足约束的情况下（可以没有约束），最优化目标函数 [e.g., MaxSAT]

计数问题：计算问题有多少个解 [e.g., #SAT]

问题分类

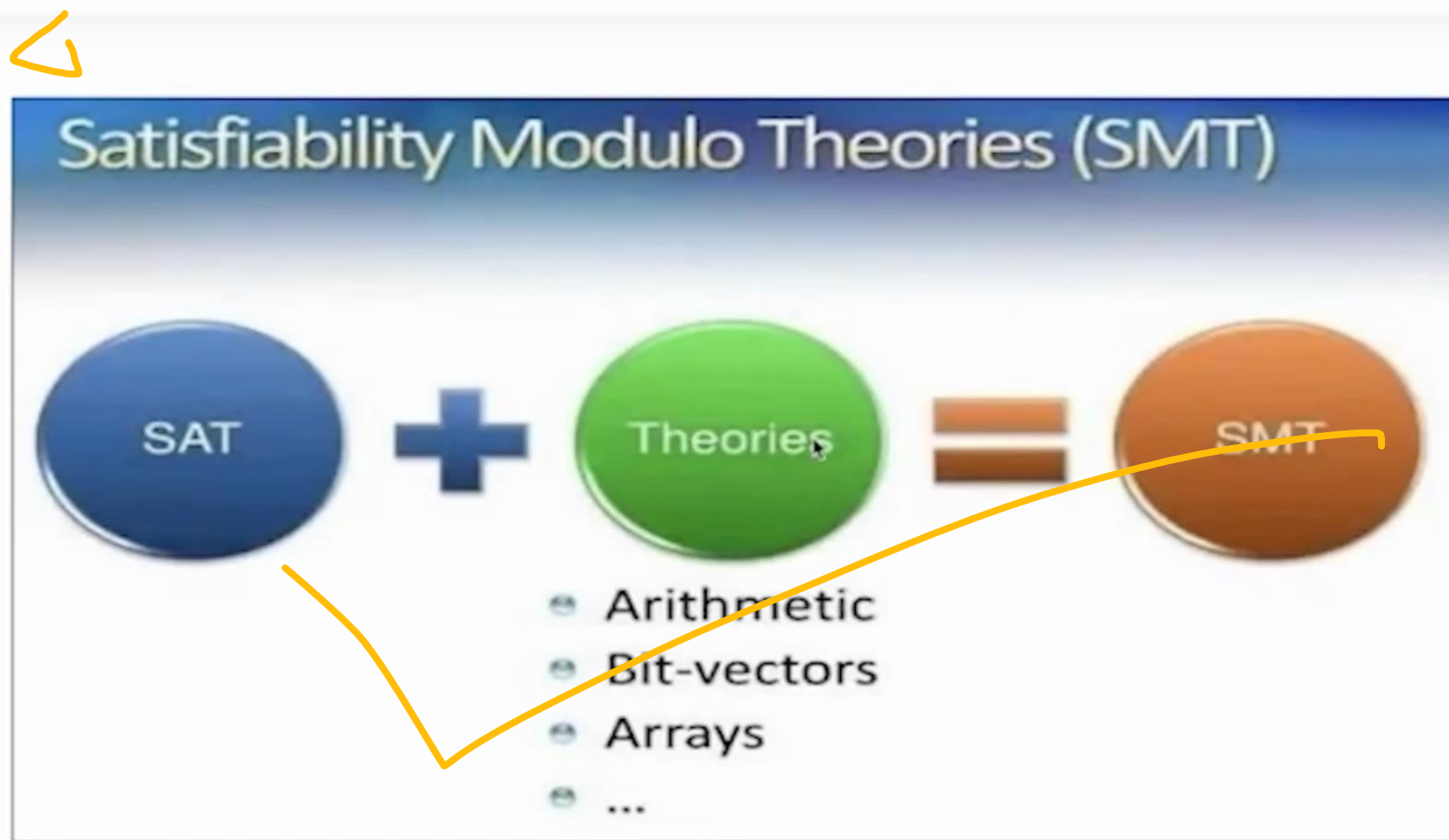
按变量类型：

- 布尔
- 整数
- 实数
- 字符串
- ...

问题分类

按变量类型：

- 布尔
- 整数
- 实数
- 字符串
- ...



问题分类

- 根据变量的类型、约束的类型、问题的目标，每个语言有不同的问题

约束模型

SAT
电路SAT
MaxSAT

(非)线性规划
二次规划
0-1规划
整数(非)线性规划
混合整数(非)线性规划

一阶逻辑可满足性问题
QBF (**带量词的布尔公式**)
SMT (不同理论的SMT)
MaxSMT
OMT (**最优模理论**)

电路SAT问题:

给定一个布尔电路 (Boolean circuit), 由逻辑门 (AND, OR, NOT, XOR ...) 以及输入、输出线组成,
问: 是否存在一种输入赋值, 使得电路输出为 1 (True) ?

SAT问题

布尔可满足性问题/命题逻辑可满足性问题，简称SAT：

给定一个命题逻辑公式，判断是否存在使其为真的赋值。

- E.g. $(x_1 \vee \neg x_2) \wedge (x_2 \vee x_3) \wedge (\neg x_1 \rightarrow x_4)$

- 合取范式(CNF):

e.g., $\varphi = (x_1 \vee \neg x_2) \wedge (x_2 \vee x_3) \wedge (x_1 \vee x_4)$
子句

识别问题的约束

- 事实/规则：
 - 当按钮被按下时，门就打开；
 - 一天有24小时；
- 必须做的：
 - 门必须有人看着
 - 每个点必须涂一个颜色
- 不能违反的：
 - 不能闯红灯
 - 两个相邻的点不能涂同一个颜色

一个例子

安排一次会议的日期，满足以下约束

- Adam can only meet on Monday or Wednesday
- Bridget cannot meet on Wednesday
- Charles cannot meet on Friday
- Darren can only meet on Thursday or Friday

建模

一个例子

安排一次会议的日期，满足以下约束

- Adam can only meet on Monday or Wednesday
- Bridget cannot meet on Wednesday
- Charles cannot meet on Friday
- Darren can only meet on Thursday or Friday

建模

$$F = (x_1 \vee x_3) \wedge (\overline{x_3}) \wedge (\overline{x_5}) \wedge (x_4 \vee x_5) \wedge \text{AtMostOne}(x_1, x_2, x_3, x_4, x_5)$$

最多只有一个为True!

一个例子

- CNF编码

$$F = (x_1 \vee x_3) \wedge (\bar{x}_3) \wedge (\bar{x}_5) \wedge (x_4 \vee x_5)$$

$$\wedge (\bar{x}_1 \vee \bar{x}_2) \wedge (\bar{x}_1 \vee \bar{x}_3) \wedge (\bar{x}_1 \vee \bar{x}_4) \wedge (\bar{x}_1 \vee x_5)$$

$$\wedge (\bar{x}_2 \vee \bar{x}_3) \wedge (\bar{x}_2 \vee \bar{x}_4) \wedge (\bar{x}_2 \vee \bar{x}_5)$$

$$\wedge (\bar{x}_3 \vee \bar{x}_4) \wedge (\bar{x}_3 \vee \bar{x}_5)$$

$$\wedge (\bar{x}_4 \vee \bar{x}_5)$$

答案: Unsatisfiable

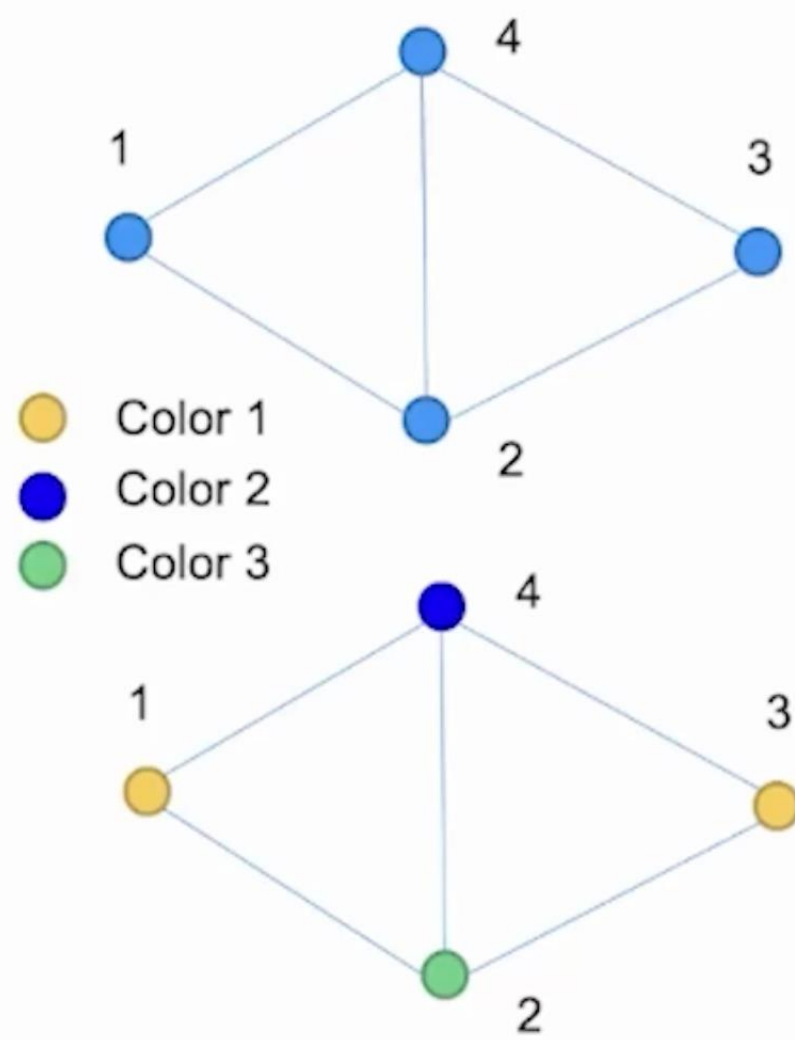
- 约束问题
- 约束模型
 - SAT
 - SMT
 - 整数线性规划
 - CSP
- 约束求解
 - 完备方法
 - 不完备方法

- 为什么我们认为SAT和线性规划是约束模型？
- 图着色问题，是不是一个约束模型？
- 一阶逻辑的可满足性问题，是不是一个约束模型？

SAT vs. 图着色

- 3-coloring problem $\rightarrow$ SAT
 - for each vertex, uses 3 variables (n vertices), $4 \times 3 = 12$ in all.

$x_{11}, x_{12}, x_{13}, x_{21}, x_{22}, x_{23}, x_{31}, x_{32}, x_{33}, x_{41}, x_{42}, x_{43}$



SAT vs. 图着色

- 3-coloring problem $\rightarrow$ SAT

- for each vertex, uses 3 variables (n vertices), $4 \times 3 = 12$ in all.

$x_{11}, x_{12}, x_{13}, x_{21}, x_{22}, x_{23}, x_{31}, x_{32}, x_{33}, x_{41}, x_{42}, x_{43}$

- For each edge, produces 3 negative 2-clause

edge1-2: $\neg x_{11} \vee \neg x_{21}, \neg x_{12} \vee \neg x_{22}, \neg x_{13} \vee \neg x_{23};$

edge1-4: $\neg x_{11} \vee \neg x_{41}, \neg x_{12} \vee \neg x_{42}, \neg x_{13} \vee \neg x_{43};$

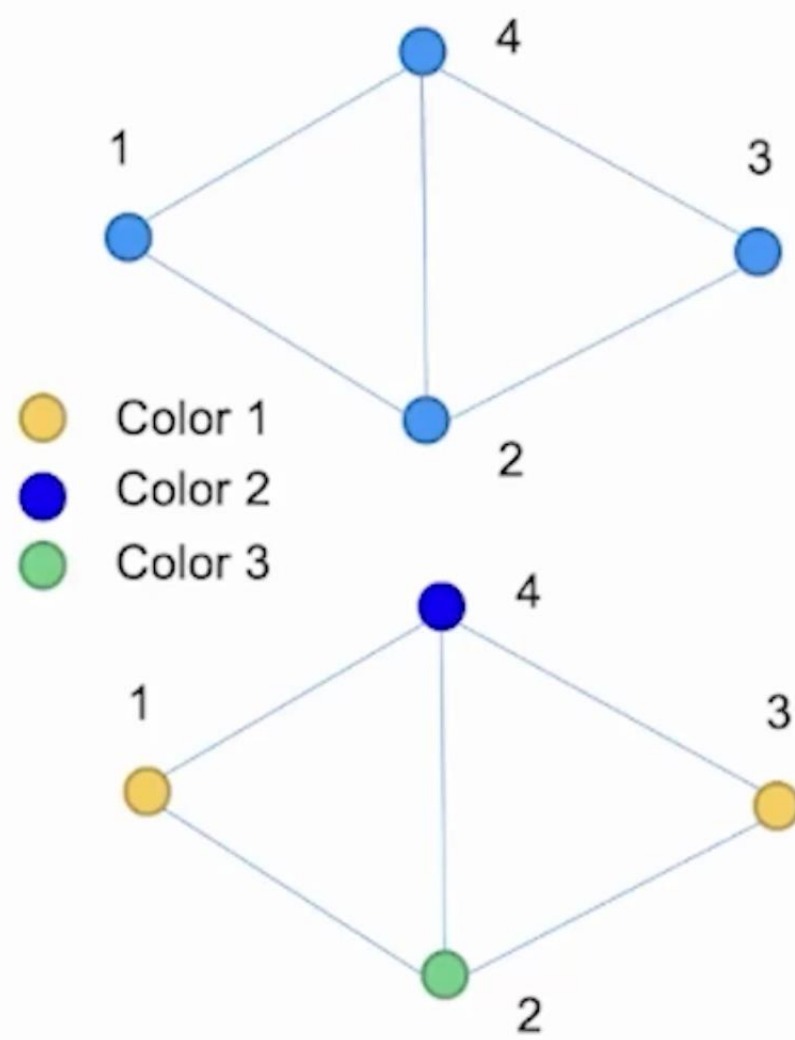
.....

- For each vertex, produces a positive k-clauses

$x_{11} \vee x_{12} \vee x_{13}, x_{21} \vee x_{22} \vee x_{23}, x_{31} \vee x_{32} \vee x_{33}, x_{41} \vee x_{42} \vee x_{43}$

- Result:

$x_{11}, \neg x_{12}, \neg x_{13}, \neg x_{21}, x_{22}, \neg x_{23}, x_{31}, \neg x_{32}, \neg x_{33}, \neg x_{41}, \neg x_{42}, x_{43}$



一阶逻辑 vs. SMT

命题逻辑

$$(A \vee B) \wedge (\neg C \vee \neg B) \wedge (\neg C \vee A)$$

一阶逻辑

$$\forall n \in \{z | z > 2, z \in \mathbb{Z}\} \neg \exists x, y, z \in \mathbb{Z} (x^n + y^n = z^n)$$

一阶逻辑 vs. SMT

命题逻辑

$$(A \vee B) \wedge (\neg C \vee \neg B) \wedge (\neg C \vee A)$$

一阶逻辑

$$\forall n \in \{z | z > 2, z \in \mathbb{Z}\} \neg \exists x, y, z \in \mathbb{Z} (x^n + y^n = z^n)$$

SMT; Satisfiability Modulo background Theories 限定背景理论的一阶逻辑

$$b+2 = c \wedge A[3] \neq A[c-b+1]$$

一个好的约束模型有什么特点？[个人观点]

- 形式化 (数学语言)
- 一般性 (方便表达很多问题)
- 可求解 (至少大部分实际情况)

- 布尔变量: $x_1, x_2, \dots$
- 文字: $x_i, \neg x_i \dots$
- 子句: 文字的析取 ($\vee$)

$$x_2 \vee x_3,$$

$$\neg x_1 \vee \neg x_3 \vee x_4$$

- 合取范式 (Conjunctive Normal Form , CNF) : 子句的合取 ($\wedge$)

e.g., $\varphi = (x_1 \vee \neg x_2) \wedge (x_2 \vee x_3) \wedge (x_1 \vee \neg x_4)$

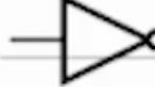
有时候, CNF也表示为子句集合, 而子句表示为文字集合:

$$\varphi = \{\{x_1, \neg x_2\}, \{x_2, x_3\}, \{x_1 \vee \neg x_4\}\}$$

任意命题逻辑公式转为CNF公式

任意命题逻辑公式转为CNF公式：Tseitin编码

- 线性时间复杂度，线性规模增长

Type	Operation	CNF Sub-expression
 AND	$C = A \cdot B$	$(\bar{A} \vee \bar{B} \vee C) \wedge (A \vee \bar{C}) \wedge (B \vee \bar{C})$
 NAND	$C = \overline{A \cdot B}$	$(\bar{A} \vee \bar{B} \vee \bar{C}) \wedge (A \vee C) \wedge (B \vee C)$
 OR	$C = A + B$	$(A \vee B \vee \bar{C}) \wedge (\bar{A} \vee C) \wedge (\bar{B} \vee C)$
 NOR	$C = \overline{A + B}$	$(A \vee B \vee C) \wedge (\bar{A} \vee \bar{C}) \wedge (\bar{B} \vee \bar{C})$
 NOT	$C = \bar{A}$	$(\bar{A} \vee \bar{C}) \wedge (A \vee C)$
 XOR	$C = A \oplus B$	$(\bar{A} \vee \bar{B} \vee \bar{C}) \wedge (A \vee B \vee \bar{C}) \wedge (A \vee \bar{B} \vee C) \wedge (\bar{A} \vee B \vee C)$

Example Φ :

$$\underbrace{(\neg s \wedge p)}_B \leftrightarrow \underbrace{((q \rightarrow r) \vee \neg p)}_C$$

yields $T(\Phi)$:

$$n_\Phi \wedge$$

$$cnf(n_\Phi \leftrightarrow (B \leftrightarrow C)) \wedge$$

$$cnf(B \leftrightarrow (\neg s \wedge p)) \wedge$$

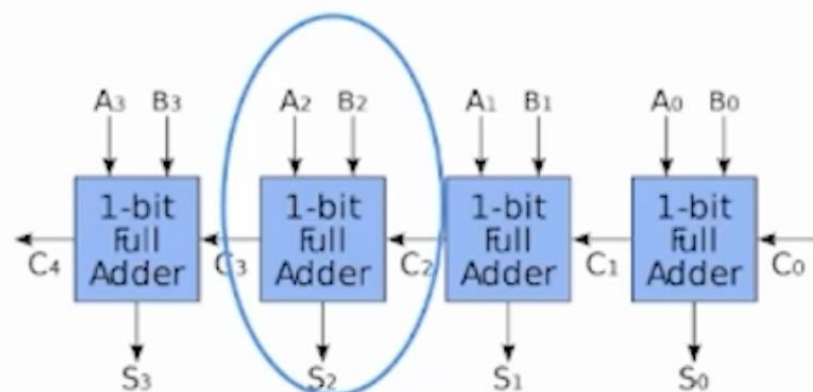
$$cnf(C \leftrightarrow (D \vee \neg p)) \wedge$$

$$cnf(D \leftrightarrow (q \rightarrow r))$$

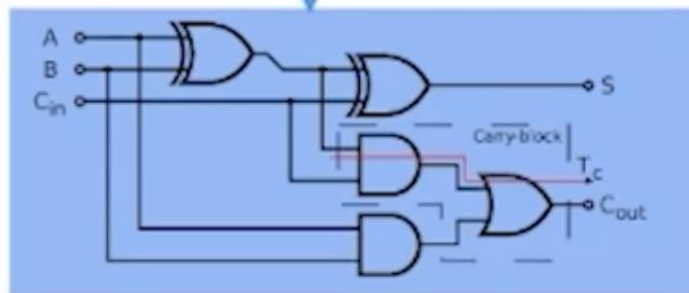
算术运算转为CNF公式

算术运算 → 基本运算单元 → 逻辑门 → SAT

$a + b$



→ CNF公式

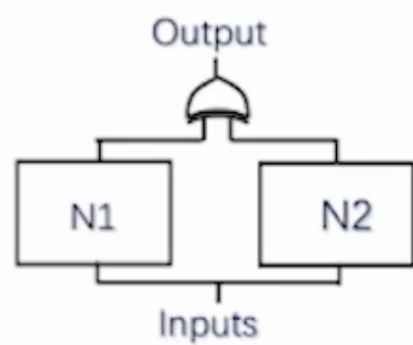


$$\begin{aligned} s &\leftrightarrow \text{XOR}(x, y, z) \quad \text{当且仅当输入中有奇数个1} \\ c &\leftrightarrow (x \wedge y) \vee (x \wedge z) \vee (y \wedge z) \end{aligned}$$

SAT应用 : EDA

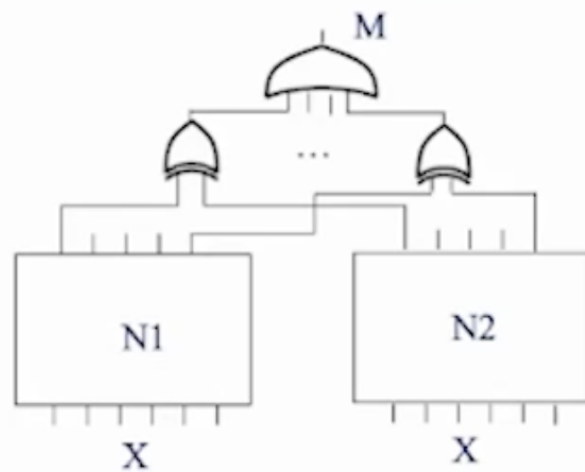
电路等价性验证

Single output



$N1 \text{ xor } N2 \rightarrow \text{CNF}$

Multi-output



SAT应用：EDA

有界模型检测（Bounded Model Checking）：检查在k步之内，是否存在一条路径，违反了指定的属性（时序逻辑公式）。

用于发现电路的
时序属性缺陷

Safety 属性：坏的事情不会发生。

LTL公式： Gp

找问题所在

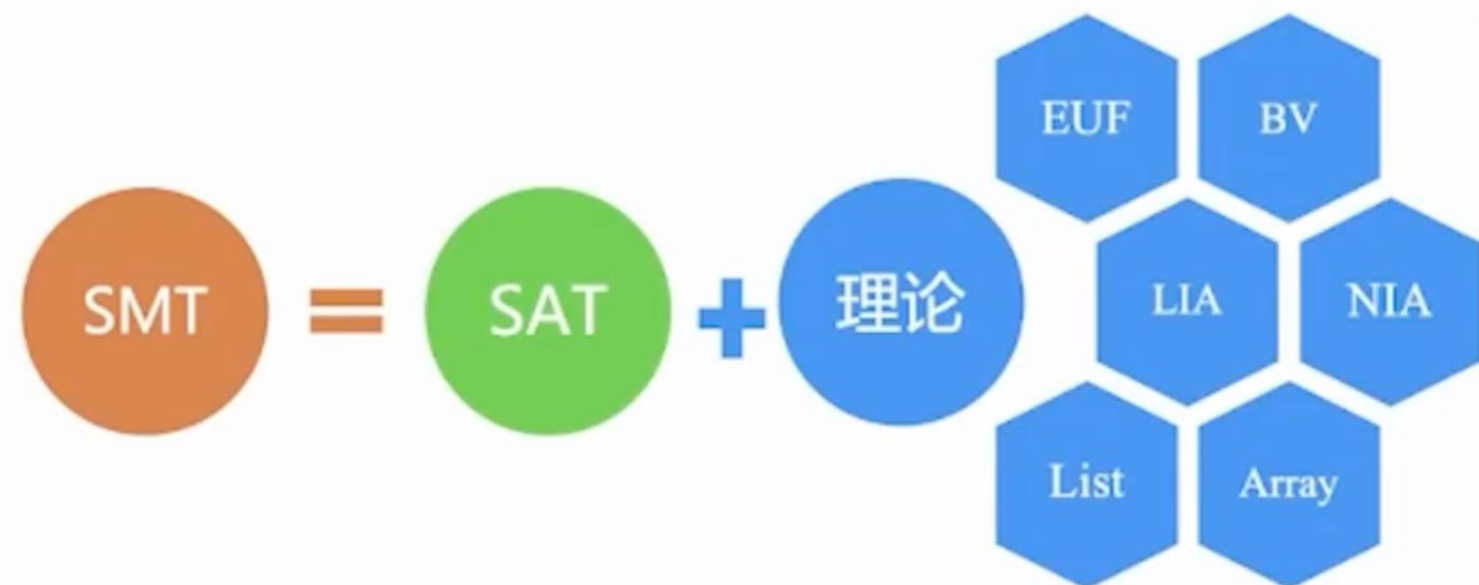
• 如果存在一条路径从初始状态达到一个状态，该状态下p不成立，则找到一个反例。

编码为SAT公式



$$\exists s_0, \dots, s_k. \quad I(s_0) \wedge \bigwedge_{i=0}^{k-1} T(s_i, s_{i+1}) \wedge \neg p(s_k)$$

限定背景理论的一阶逻辑判定



$$y^2 + 2xy < 100 \vee f(y) < 30 \rightarrow \neg(x - y < 30) \wedge x \leq 60$$

Example:

- $a = b + 2 \wedge A = \text{write}(B, a + 1, 4) \wedge (\text{read}(A, b + 3) = 2 \vee f(a - 1) \neq f(b + 1))$
- 命题骨架 $\text{PS}_\Phi = y_1 \wedge y_2 \wedge (y_3 \vee y_4)$
- $y_1: a = b + 2$
- $y_2: A = \text{write}(B, a + 1, 4)$
- $y_3: \text{read}(A, b + 3) = 2$
- $y_4: f(a - 1) \neq f(b + 1)$

SMT应用：软件验证

Program 12.2.1 Reading blocks from an array

```
1 void ReadBlocks(int data[], int cookie)
2 {
3     int i = 0;
4     while (true)
5     {
6         int next;
7         next = data[i];
8         if (!(i < next && next < N)) return;
9         i = i + 1;
10        for (; i < next; i = i + 1) {
11            if (data[i] == cookie)
12                i = i + 1;
13            else
14                Process(data[i]);
15        }
16    }
17 }
```

$\text{assert}(0 \leq i \ \&\& \ i < N)$

$i_1 = 0$ $\wedge$
 $next_1 = data_0[i_1]$ $\wedge$
 $(i_1 < next_1 \wedge next_1 < N_0)$ $\wedge$
 $i_2 = i_1 + 1$ $\wedge$
 $i_2 < next_1$ $\wedge$
 $data_0[i_2] = cookie_0$ $\wedge$
 $i_3 = i_2 + 1$ $\wedge$
 $i_4 = i_3 + 1$ $\wedge$
 $\neg(i_4 < next_1)$ $\wedge$
 $\neg(0 \leq i_4 \wedge i_4 < N_0)$

$\neg(0 \leq i_4 \ \&\& \ i_4 < N_0)$

$data_0 =$
 $\langle 2, 6, 5 \rangle$

SMT应用：软件测试

动态符号执行

```
int foo(int v)
{
    return 2*v;
}
void test_me(int x, int y)
{
    int z = foo(y);
    if (z == x)
        if (x > y+10)
            ERROR;
}
```



第1次执行

实际执行

符号执行

实际状态

符号状态

路径条件

x = 22
y = 7

x = x₀
y = y₀

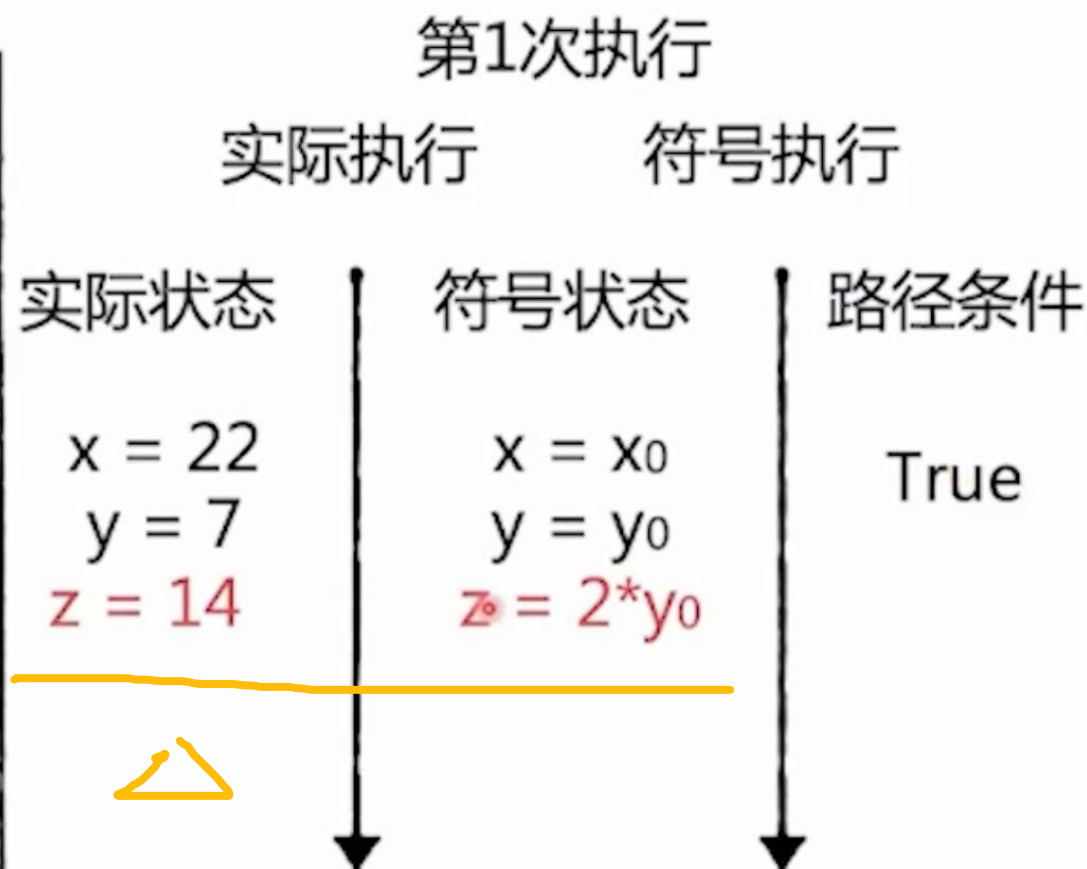
True



SMT应用：软件测试

动态符号执行

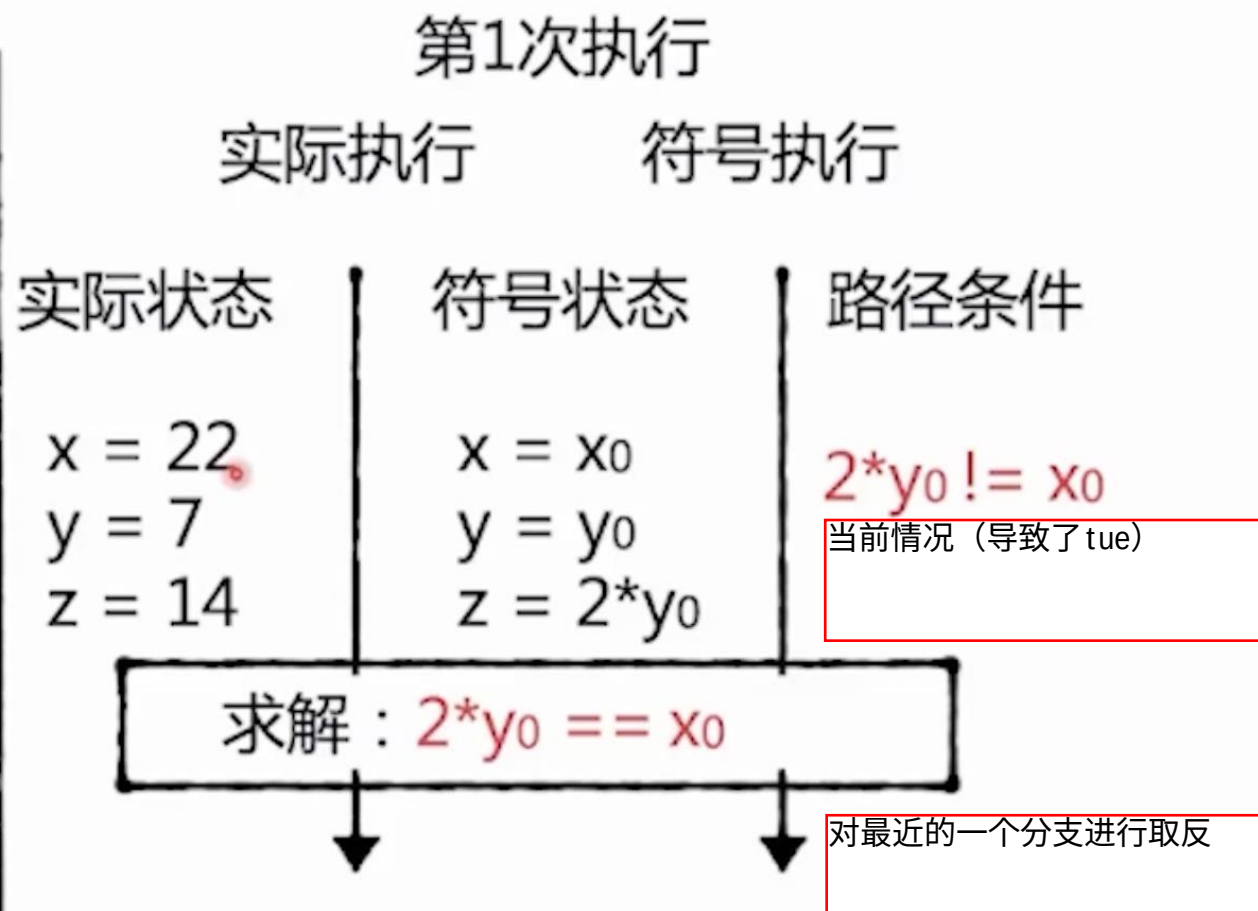
```
int foo(int v)
{
    return 2*v;
}
void test_me(int x, int y)
{
    int z = foo(y);
    if (z == x)
        if (x > y+10)
            ERROR;
}
```



SMT应用：软件测试

动态符号执行

```
int foo(int v)
{
    return 2*v;
}
void test_me(int x, int y)
{
    int z = foo(y);
    if (z == x)
        if (x > y+10)
            ERROR;
}
```



SMT应用：软件测试

动态符号执行

```
int foo(int v)
{
    return 2*v;
}
void test_me(int x, int y)
{
    int z = foo(y);
    if (z == x)
        if (x > y+10)
            ERROR;
}
```



第1次执行

实际执行

符号执行

实际状态

符号状态

路径条件

x = 22
y = 7
z = 14

x = x₀
y = y₀
z = 2*y₀

2*y₀ != x₀

求解：2*y₀ == x₀

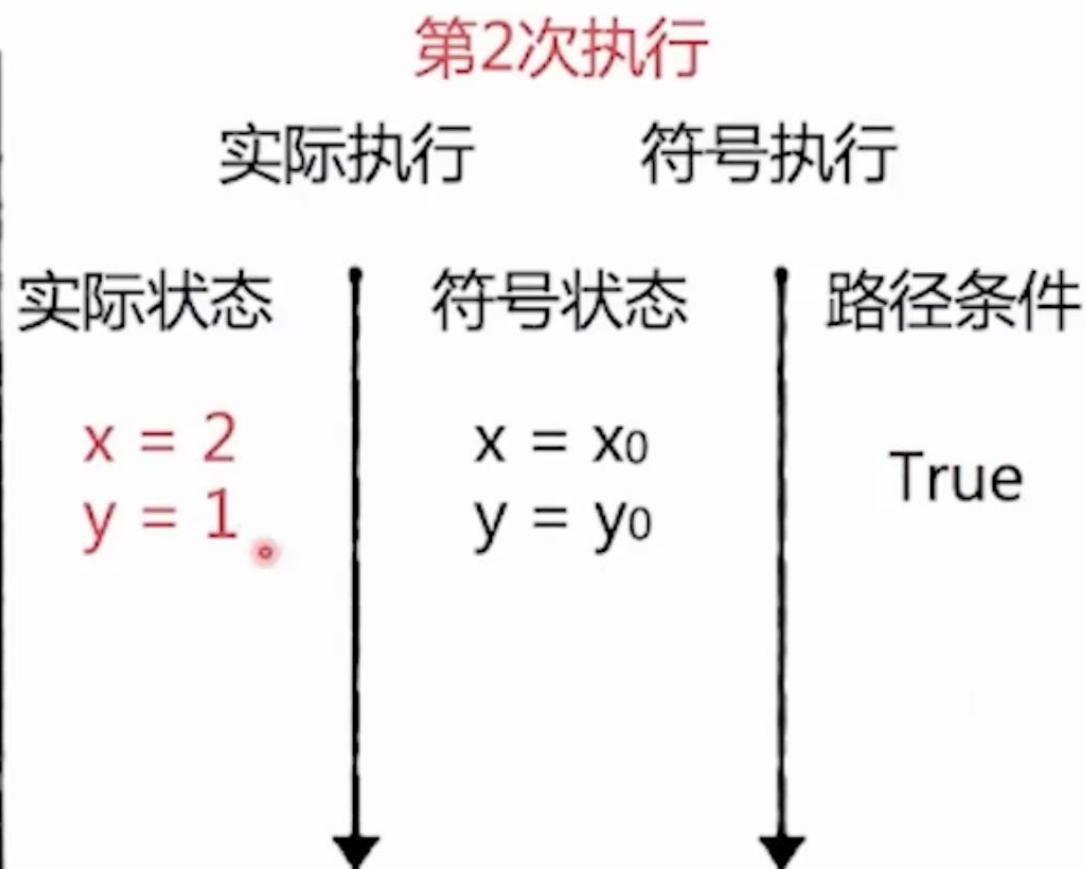
结果：x₀ == 2, y₀ == 1

计算出新的输入，接着代入尝试

SMT应用：软件测试

动态符号执行

```
int foo(int v)
{
    return 2*v;
}
void test_me(int x, int y)
{
    int z = foo(y);
    if (z == x)
        if (x > y+10)
            ERROR;
}
```



SMT应用：软件测试

动态符号执行

```
int foo(int v)
{
    return 2*v;
}
void test_me(int x, int y)
{
    int z = foo(y);
    if (z == x)
        if (x > y+10) ←
            ERROR;
}
```

第2次执行

实际执行

符号执行

实际状态

符号状态

路径条件

$x = 2$
 $y = 1$
 $z = 2$

$x = x_0$
 $y = y_0$
 $z = 2*y_0$

$2*y_0 == x_0$
 $\wedge$
 $x_0 \leq y_0 + 10$

SMT应用：软件测试

动态符号执行

```
int foo(int v)
{
    return 2*v;
}
void test_me(int x, int y)
{
    int z = foo(y);
    if (z == x)
        if (x > y+10) ←
            ERROR;
}
```

第2次执行

实际执行

符号执行

实际状态

符号状态

路径条件

$x = 2$
 $y = 1$
 $z = 2$

$x = x_0$
 $y = y_0$
 $z = 2*y_0$

$2*y_0 == x_0$
 $\wedge$
 $x_0 <= y_0 + 10$

求解： $2*y_0 == x_0 \wedge x_0 > y_0 + 10$

结果： $x_0 == 22, y_0 == 11$

还是对最近的一个情况取反
重新计算新的输入

SMT应用：软件测试

动态符号执行

```
int foo(int v)
{
    return 2*v;
}
void test_me(int x, int y)
{
    int z = foo(y);
    if (z == x)
        if (x > y+10) ←
            ERROR;
}
```

第3次执行

实际执行

符号执行

实际状态

符号状态

路径条件

x = 22
y = 11
z = 22

x = x₀
y = y₀
z = 2*y₀

2*y₀ == x₀
∧
x₀ > y₀ + 10

SMT应用：软件测试

动态符号执行

```
int foo(int v)
{
    return 2*v;
}
void test_me(int x, int y)
{
    int z = foo(y);
    if (z == x)
        if (x > y+10) ←
            ERROR;
}
```

触发Error

第3次执行

实际执行

符号执行

实际状态

符号状态

路径条件

x = 22
y = 11
z = 22

x = x₀
y = y₀
z = 2*y₀

$2*y_0 == x_0$
 $\wedge$
 $x_0 > y_0 + 10$

SMT应用：软件测试

动态符号执行

```
int foo(int v)
{
    return 2*v;
}
void test_me(int x, int y)
{
    int z = foo(y);
    if (z == x)
        if (x > y+10) 
            ERROR;
}
```

触发Error

第3次执行

实际执行

符号执行

实际状态

符号状态

路径条件

x = 22
y = 11
z = 22

x = x₀
y = y₀
z = 2*y₀

$2*y_0 == x_0$
 $\wedge$
 $x_0 > y_0 + 10$

输入：x₀ == 22, y₀ == 11

线性规划

- 标准型:

$$\min z = c'x$$

$$\text{s. t. } \begin{cases} Ax = b \\ x \geq 0 \end{cases}$$

c is a weight vector; **x** represents the vector of variables

A represents a matrix; **b** is a column vector

- (实数) 线性规划
- 整数线性规划
- 混合整数规划

线性规划

从一般线性约束转为标准型号

- $a'x = b$ $\rightarrow$ *standard form*
- $a'x \geq b$ $\rightarrow a'x - x_s = b, \text{ where } x_s \geq 0$
- $a'x \leq b$ $\rightarrow a'x + x_s = b, \text{ where } x_s \geq 0$
- $x_i \leq 0$ $\rightarrow y_i = -x_i$
- $x_i \in R$ $\rightarrow x_i = z_1 - z_2, \text{ where } z_1, z_2 \geq 0$
- $\max c'x$ $\rightarrow \min -c'x$

核心目标：将不等式转换成等式
通过引入变量来实现，相当于原先的不等号变成了新变量的不等约束

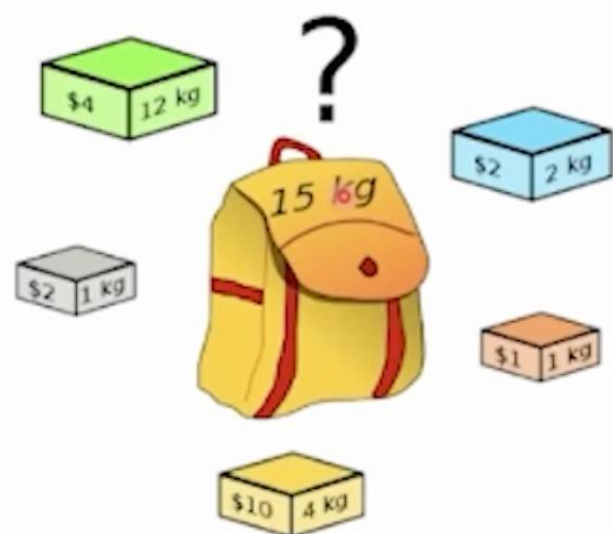
Example:

$$\begin{aligned} & \max 100x_1 + 200x_2 \\ & \text{s. t. } \begin{cases} 30x_1 + 40x_2 \leq 500 \\ 40x_1 + 60x_2 \leq 700 \\ x_1, x_2 \geq 0 \end{cases} \end{aligned}$$

$$\begin{aligned} & \min -(100x_1 + 200x_2) \\ & \rightarrow \text{s. t. } \begin{cases} 30x_1 + 40x_2 + c_1 = 500 \\ 40x_1 + 60x_2 + c_2 = 700 \\ x_1, x_2, c_1, c_2 \geq 0 \end{cases} \end{aligned}$$

整数线性规划：资源分配问题

- 背包问题

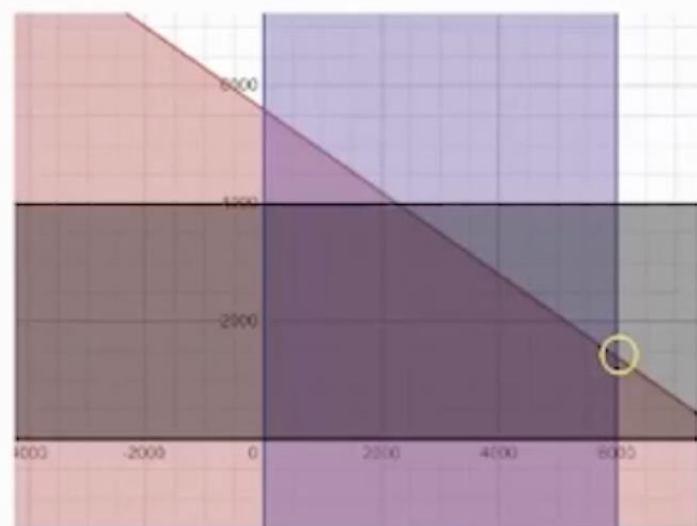


$$\begin{cases} \text{Max} \sum_{i=1}^N p_i x_i \\ \text{s.t} \sum_{i=1}^N w_i x_i \leq C; \\ x_i \in \{0,1\}; \forall i = 1, \dots, N \end{cases}$$

$$\begin{cases} \text{Max} \sum_{k=1}^M \sum_{i=1}^N p_i x_{ik} \\ \text{s.t} \sum_{i=1}^N w_{ij}^k x_{ik} \leq C_j^k; \forall j = 1, \dots, d; \forall k = 1, \dots, M; \\ \sum_{k=1}^M x_{ik} \leq 1; \forall i = 1, \dots, N; \\ x_{ik} \in \{0,1\}; \forall i = 1, \dots, N; k = 1, \dots, M; \end{cases}$$

混合整数规划：生产规划

- 生产规划：某钢材加工厂生产钢板和钢丝两种商品，其中生产钢板的速度是每小时200吨，生产钢丝的速度是每小时140吨，工厂可用的加工时间为40小时。每吨钢板的利润是25美元，每吨钢丝的利润是30美元。生产钢板和钢丝的数量上限分别为6000和4000。该钢材加工厂为了最大化利润，应该如何规划生产？



$$\text{Maximize } 25 X_B + 30 X_C$$

$$\text{Subject to } (1/200) X_B + (1/140) X_C \leq 40$$

$$0 \leq X_B \leq 6000$$

$$0 \leq X_C \leq 4000$$

$$\text{最优解：} X_B = 6000, X_C = 1400$$

$$\text{Obj} = 192000$$

CSP目前已经支持超过400种约束关系，是一种建模友好的语言

- Constraint Satisfaction Problem：中文一般称为约束可满足问题，简称CSP
- 一个CSP被看做是一个三元组 $\langle V, D, C \rangle$ ，其中：
 - V是变量的集合
 - D是每个变量的值域
 - C是一组约束
- 问题的目标是：求一个对变量的赋值，该赋值满足C中所有的约束

全局约束

- alldifferent(X)
- allequal(X)
- atleast(N,X,value)
- regular(X,DFA)
//accepted by a DFA

CSP : 求解谜题

- 数字谜：SEND+MORE=MONEY, 每个字母代表一个0~9的数字，各不相同。

```
SEND-MORE-MONEY ≡ \[DOWNLOAD\]
include "alldifferent.mzn";

var 1..9: S;
var 0..9: E;
var 0..9: N;
var 0..9: D;
var 1..9: M;
var 0..9: O;
var 0..9: R;
var 0..9: Y;

constraint
    1000 * S + 100 * E + 10 * N + D
    + 1000 * M + 100 * O + 10 * R + E
    = 10000 * M + 1000 * O + 100 * N + 10 * E + Y;

constraint alldifferent([S,E,N,D,M,O,R,Y]);

solve satisfy;

output [ "    ", show(S), show(E), show(N), show(D), "\n",
        "+ ", show(M), show(O), show(R), show(E), "\n",
        "= ", show(M), show(O), show(N), show(E), show(Y), "\n"];
```

高级约束建模语言MiniZinc:

MiniZinc 本身不求解问题，而是将模型编译为 FlatZinc (.fzn)，再交给底层求解器（如 Gecode、Chuffed、OR-Tools、CPLEX、SCIP 等）。

CSP : 调度问题

调度场景：移动家具

- 每移动一个家具 i ，需要的工人和推车数量： $handler[i]$ 和 $trolleys[i]$
- 如何安排移动每个家具的开始时刻 $start[i]$ ，使得任务尽早完成？
- **全局约束**：任何时刻使用的工人和推车都不会超过其原有数量。

```
include "cumulative.mzn";

enum OBJECTS;
array[OBJECTS] of int: duration; % duration to move
array[OBJECTS] of int: handlers; % number of handlers required
array[OBJECTS] of int: trolleys; % number of trolleys required

int: available_handlers;
int: available_trolleys;
int: available_time;

array[OBJECTS] of var 0..available_time: start;
var 0..available_time: end;

constraint cumulative(start, duration, handlers, available_handlers);
constraint cumulative(start, duration, trolleys, available_trolleys);

constraint forall(o in OBJECTS)(start[o] + duration[o] <= end);
solve minimize end;

output [ "start = \(start)\nend = \(end)\n"];
```

$start[i]$ —— 第 i 个家具的开始时间
 end —— 所有任务完成的最晚时间 (makespan)

所有任务都必须在 end 之前完成

约束模型的转换

SAT是整数线性规划的特殊情况：

$$l_1 \vee l_2 \vee \dots \vee l_n \Leftrightarrow l_1 + l_2 + \dots + l_n \geq 1, \\ l_i \in \{x_i, \neg x_i\}, x_i \in \{0, 1\}$$

当然，SAT也是SMT的特殊情况

算术表达式可以用布尔变量进行展开

整数规划，算术理论SMT都可以用积极编码方法表达为CNF公式

约束模型的转换

用整数线性规划表达命题逻辑关系：

if ($P = 1$) then $A < B$ else $B < A$.

$\implies$ if ($P = 1$) then $(A - B < 0)$ else $(B - A < 0)$.

$\implies$

$$A - B < M1 * (1 - P) \quad \dots\dots\dots (1)$$

$$B - A < M2 * P \quad \dots\dots\dots (2)$$

- $P = \{0, 1\}$, $M1$ 和 $M2$ 是足够大的常数

所以m的系数不为0的时候，就相当于没有约束

If $P=1$, then $A < B$ and constraint (2) is redundant.

If $P=0$, then $B < A$ and constraint (1) is redundant.

$\implies$

$$A - B \leq M1 * (1 - P) - 1 \quad \dots\dots\dots (3)$$

$$B - A \leq M2 * P - 1 \quad \dots\dots\dots (4)$$

ILP 通常只有 $\leq$ ，没有严格 $<$ 又因为 A 、 B 是整，所以最后减1

约束模型的选择

很多情况下，同一个问题可以用不同的约束模型表达

- 比如调度问题

采取哪个约束模型，考虑的主要因素有

- 表达的便易性
- 求解器的性能
- 有时候需要配合不同模型求解器

- 约束问题
- 约束模型
 - SAT
 - SMT
 - 整数线性规划
 - CSP
- 约束求解
 - 完备方法
 - 不完备方法

“求解器”是什么？

求解器：求解一个约束模型的程序。

其形象可能是这样：



也可能是这样：



但应该不是这样：



“启发式” 是什么？

- “In fact, if the algorithm is complicated enough, proving things about it (i.e., whether or not it works) becomes very difficult for even the best experts. ...Thus not all algorithms that have been designed have been analyzed. ... The algorithms we study today are called heuristics: for most of them **we know that they do not work on worst-case instances, but there is good evidence that they work very well on many instances of practical interest**. Explaining this discrepancy theoretically is an interesting and challenging open problem.”

--- Sanjeev Arora (a famous theoretical computer scientist)

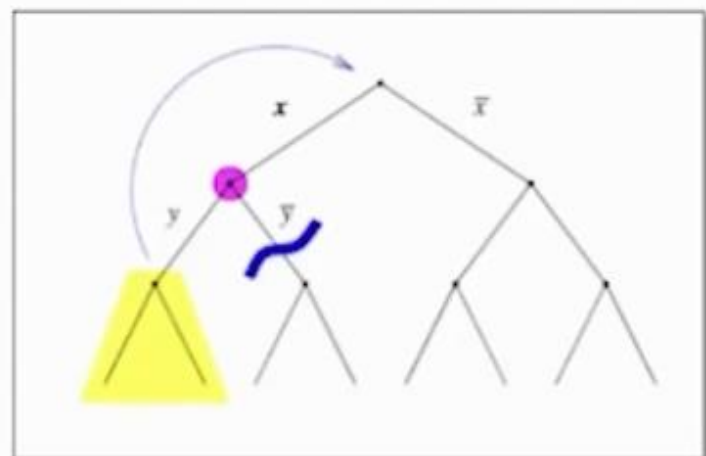
- 启发式方法
- 启发式

两类方法

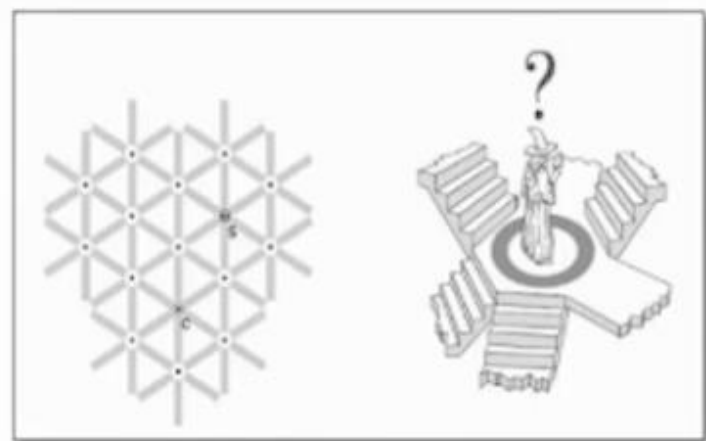
约束求解

完备算法/精确算法：算法结束时确保问题最优解，往往难以解决大规模问题。

不完备算法/启发式方法：不保证最优解，争取在短时间内返回解（近似解）。



系统搜索



局部搜索

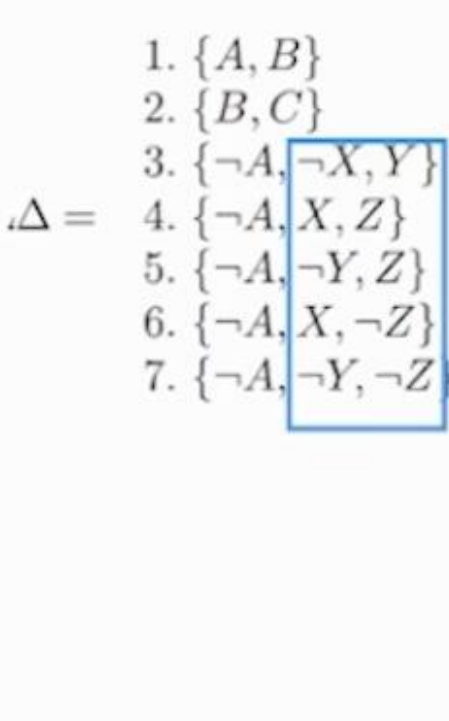
完备算法(Complete Algorithms)

基于完备算法的约束求解器

- 算法框架
- 决策启发式
- 推理技术

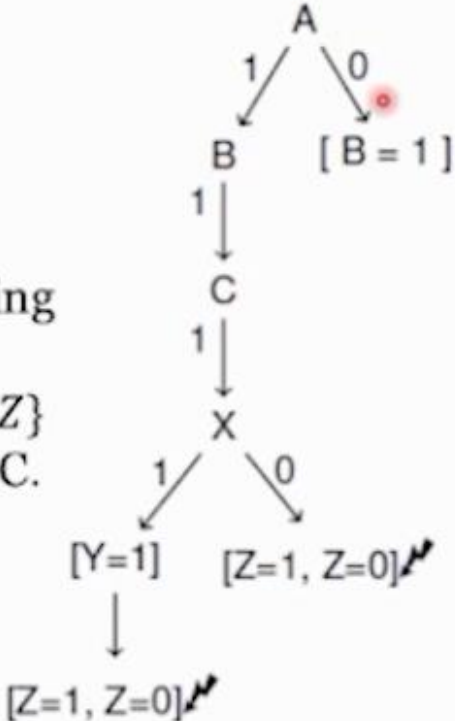
非时序回溯

所谓时序就是遍历的时间顺序
非时序回溯，就是不按照时间顺序回溯
直接backjump



时序回溯

The first two conflicting clauses
 $\{\neg A, \neg X, Y\}, \{\neg A, X, \neg Z\}$
do not involve B and C.



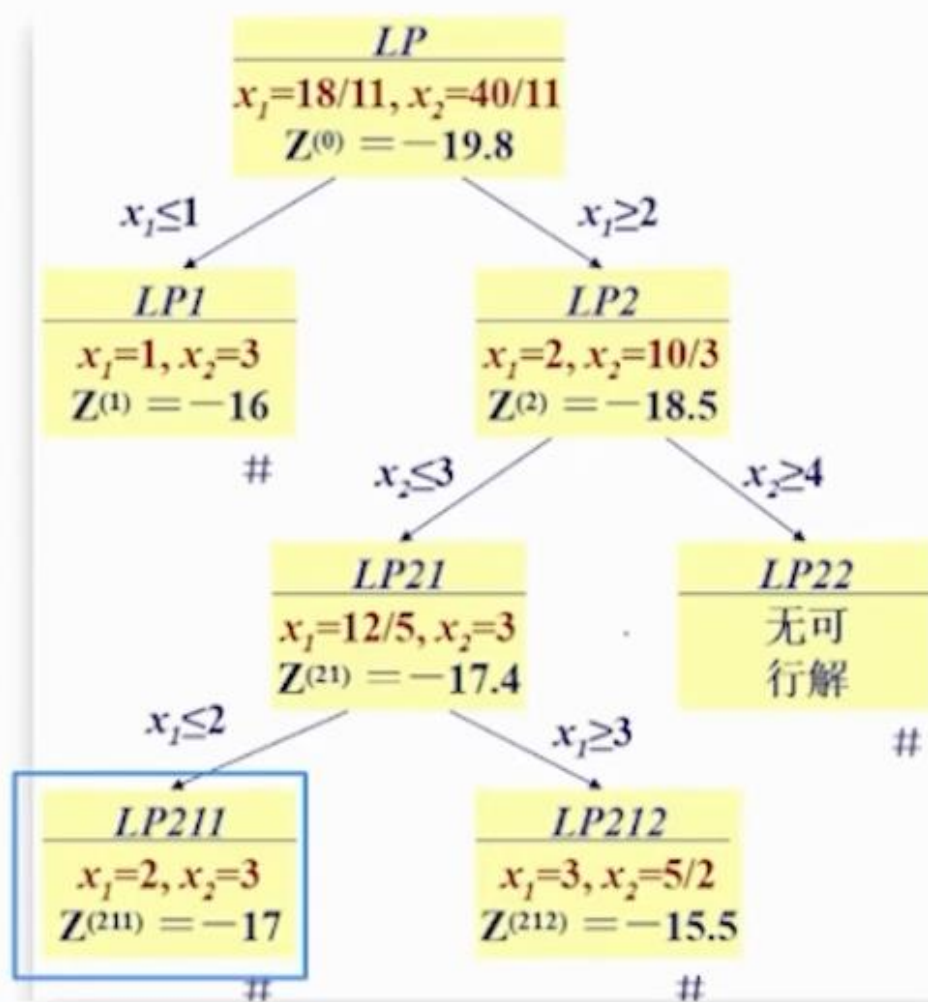
非时序回溯

分支限界

分支限界法一般用于优化问题

$$\min W = -x_1 - 5x_2$$

$$\text{s.t.} \begin{cases} x_1 - x_2 \geq -2 \\ 5x_1 + 6x_2 \leq 30 \\ x_1 \leq 4 \\ x_1, x_2 \geq 0 \text{ 并且为整数} \end{cases}$$



完备算法(Complete Algorithms)

基于完备算法的约束求解器

- 算法框架
- 决策启发式
- 推理技术

分支启发式

Fail fast 原则

- SAT : Variable State Independent Decaying Sum (VSIDS) 变量状态独立衰减和
 - 每个变量维护一个活跃分数,挑选最大分数的变量
 - 初始化为其出现次数
 - 每次产生一个新的冲突子句 : 该子句的所有变量增加分数
 - 周期性地对所有分数都乘以一个小于1的正常数 // 忘记之前的效果

类比操作系统中换页操作:

LFU (Least Frequently Used, 最少使用频率)

思想: 换出访问次数最少的页。

实现:

- 为每个页维护一个计数器, 记录累计访问次数;
- 换页时选访问次数最少的。

分支启发式

Fail fast 原则

- CSP分支变量启发式： $dom/wdeg$
 - 每个变量维护一个分数 $dom/wdeg$, dom 为变量的实时可行域大小, $wdeg$ (加权重) 衡量了变量参与冲突的次数, 挑选最小分数的变量
 - $Wdeg$ 初始化为变量出现的约束个数, 也即变量的度数
 - 每次遇到一个新的冲突约束: 该约束的所有变量的 $wdeg$ 增加1
 - 周期性地遗忘 $wdeg$

分支启发式

分支限界（线性整数规划）尽量优化分支后的下界，加速剪枝

- Strong Branch
 - 对每个变量 v ，计算分支它得到的两个子问题的下界（通过LP松弛）： $LB(v_{up}), LB(v_{down})$ ，原问题下界为 $LB(v) = \min\{LB(v_{up}), LB(v_{down})\}$
 - 选择产生的下界最佳的变量进行分支
- Pseudo Cost Branch
 - 选择历史上单位变化对LP松弛目标函数的增大量最大的变量进行分支（变化率）
 - 需要前序分支策略来作为历史信息参考（一般为Strong Branch）
- 综合策略：实际常用的多为综合策略，如Reliability Branching（Pseudo Cost Branch与Strong Branch结合）

完备算法(Complete Algorithms)

基于完备算法的约束求解器

- 算法框架
- 决策启发式
- 推理技术

约束传播

SAT : 布尔约束传播 (单元传播)

Example:

	x_1	T		x_1	sat
	x_2			$\bar{x}_1 \vee \bar{x}_2$	
$Vars:$	x_3		$clauses:$	$\bar{x}_1 \vee \bar{x}_3$	
	x_4			$x_2 \vee \bar{x}_3 \vee \bar{x}_4$	
	x_5			$\bar{x}_1 \vee x_4 \vee x_5$	
	x_6			$x_2 \vee x_4 \vee x_6$	

约束传播

SAT : 布尔约束传播 (单元传播)

Example:

	x_1	T		x_1	sat
	x_2			$\bar{x}_1$	$\bar{x}_2$
$Vars:$	x_3		$clauses:$	$\bar{x}_1$	x_3
	x_4			x_2	$\bar{x}_3 \vee \bar{x}_4$
	x_5			$\bar{x}_1$	$x_4 \vee x_5$
	x_6			x_2	$x_4 \vee x_6$

约束传播

CSP : 弧一致性

每个变量的值域中的所有值都满足它的所有二元约束。

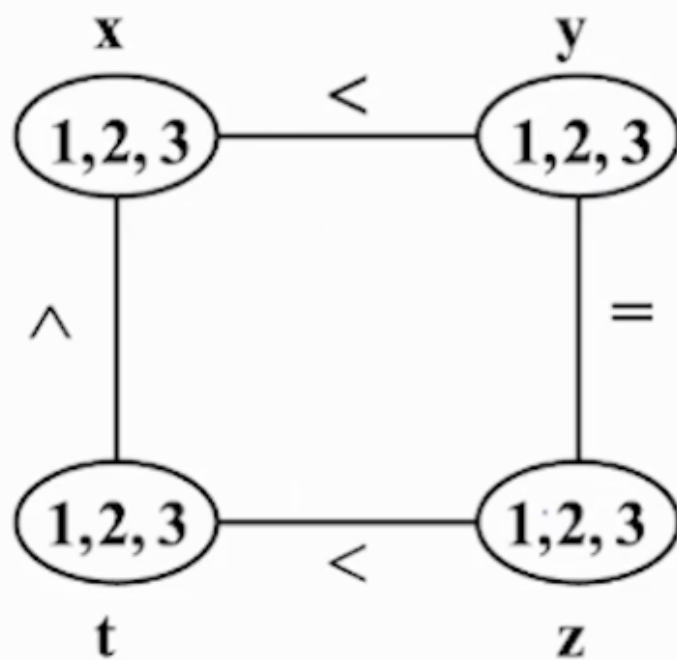
$$1 \leq x, y, z, t \leq 3$$

$$x < y$$

$$y = z$$

$$t < z$$

$$x < t$$



约束传播

CSP：弧一致性

每个变量的值域中的所有值都满足它的所有二元约束。

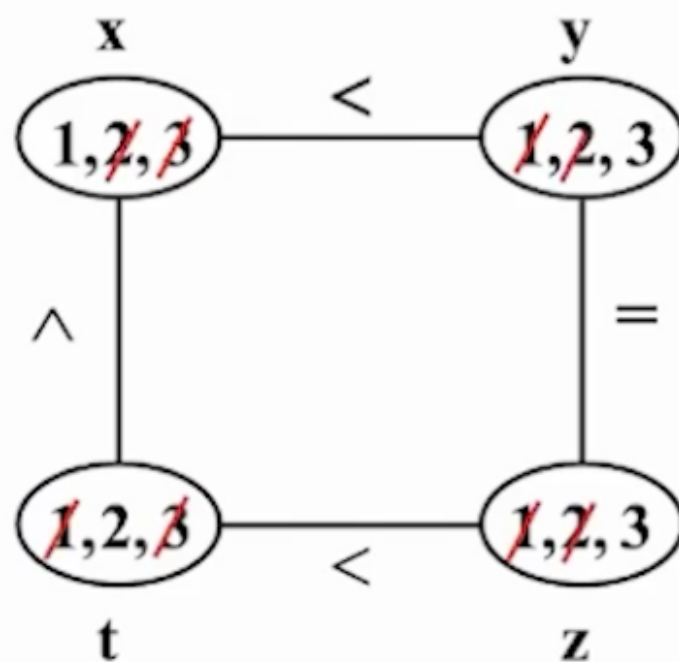
$$1 \leq x, y, z, t \leq 3$$

$$x < y$$

$$y = z$$

$$t < z$$

$$x < t$$



约束传播

域传播：通过分析约束和当前变量域，进一步缩小变量域

整数线性约束：

$$3x_1 + x_2 + x_3 \leq 4,$$

$$x_1 \in [0,2], x_2, x_3 \in [0,1]$$

域传播： $x_2, x_3 \in [0,1] \rightarrow x_1 \in [0,1]$

CSP全局约束：alldifferent (x_1, x_2, x_3, x_4)

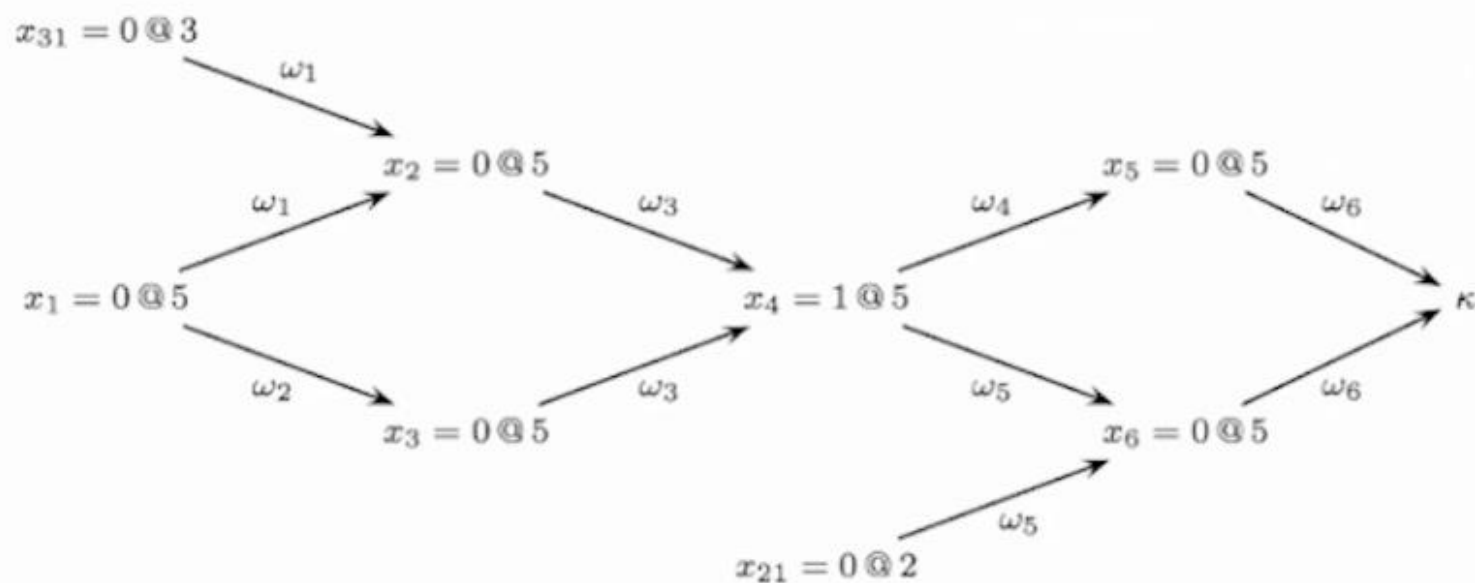


$$Dom(x_1) = Dom(x_2) = \{g, b\} \rightarrow Dom(x_3) = Dom(x_4) = \{r, y\}$$

冲突分析

SAT的冲突分析

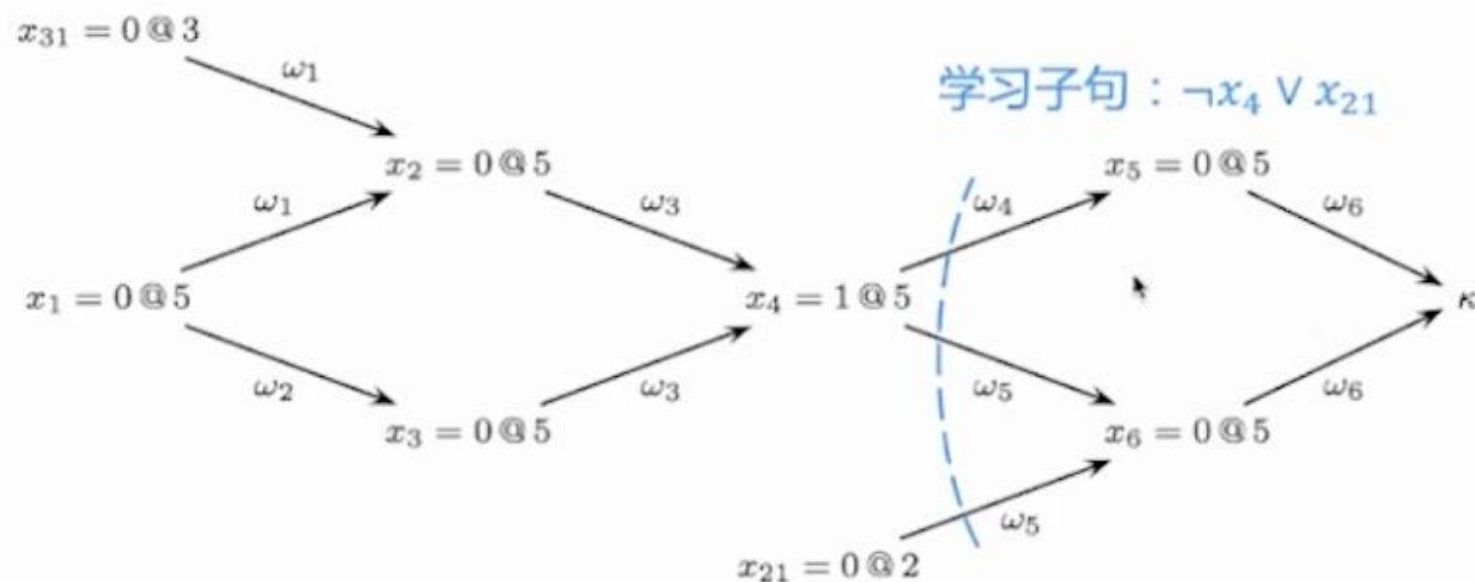
$$\begin{aligned}\varphi_1 &= \omega_1 \wedge \omega_2 \wedge \omega_3 \wedge \omega_4 \wedge \omega_5 \wedge \omega_6 \\ &= (x_1 \vee x_{31} \vee \neg x_2) \wedge (x_1 \vee \neg x_3) \wedge (x_2 \vee x_3 \vee x_4) \wedge \\ &\quad (\neg x_4 \vee \neg x_5) \wedge (x_{21} \vee \neg x_4 \vee \neg x_6) \wedge (x_5 \vee x_6)\end{aligned}$$



冲突分析

SAT的冲突分析

$$\begin{aligned}\varphi_1 &= \omega_1 \wedge \omega_2 \wedge \omega_3 \wedge \omega_4 \wedge \omega_5 \wedge \omega_6 \\ &= (x_1 \vee x_{31} \vee \neg x_2) \wedge (x_1 \vee \neg x_3) \wedge (x_2 \vee x_3 \vee x_4) \wedge \\ &\quad (\neg x_4 \vee \neg x_5) \wedge (x_{21} \vee \neg x_4 \vee \neg x_6) \wedge (x_5 \vee x_6)\end{aligned}$$

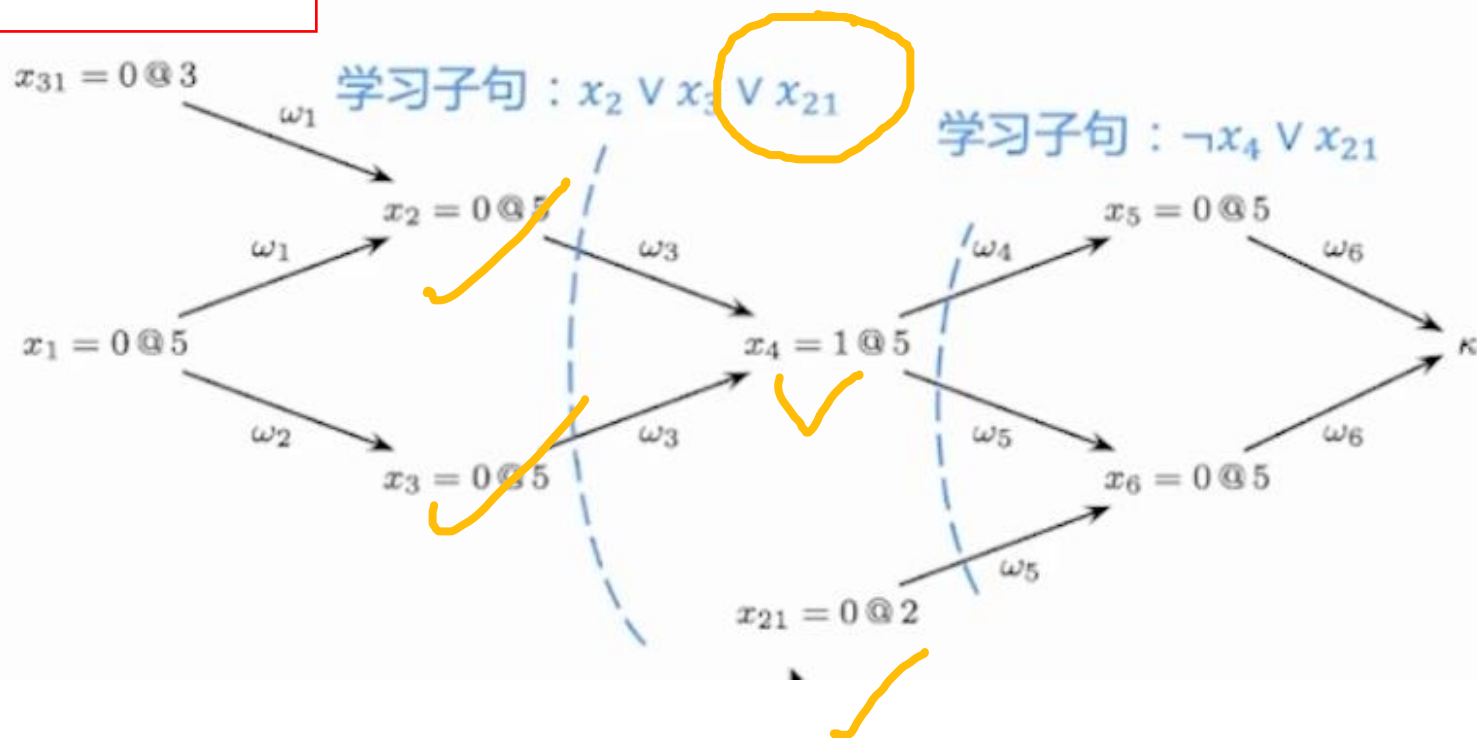


冲突分析

SAT的冲突分析

$$\begin{aligned}\varphi_1 &= \omega_1 \wedge \omega_2 \wedge \omega_3 \wedge \omega_4 \wedge \omega_5 \wedge \omega_6 \\ &= (x_1 \vee x_{31} \vee \neg x_2) \wedge (x_1 \vee \neg x_3) \wedge (x_2 \vee x_3 \vee x_4) \wedge \\ &\quad (\neg x_4 \vee \neg x_5) \wedge (x_{21} \vee \neg x_4 \vee \neg x_6) \wedge (x_5 \vee x_6)\end{aligned}$$

取反，做析取



冲突分析

整数线性规划的冲突分析

- 约束冲突：指某些变量取某些赋值的时候，问题约束之间矛盾

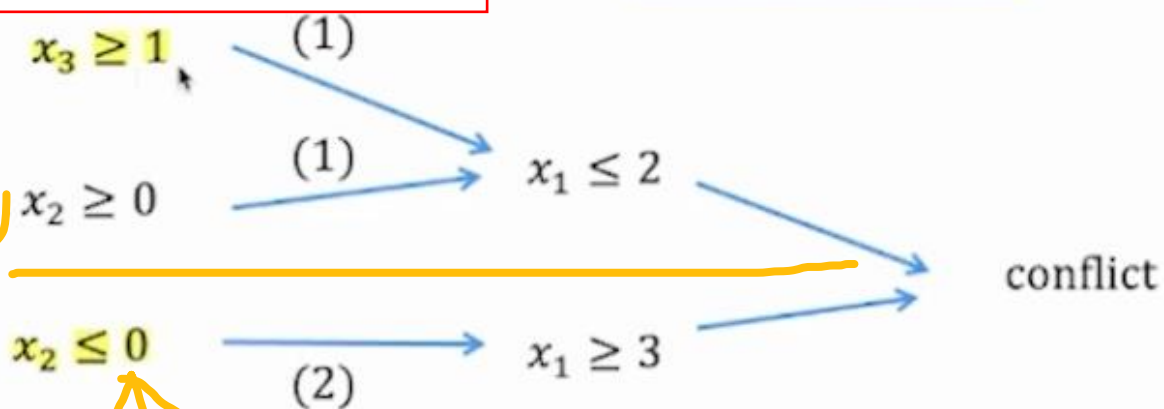
原始约束

$$\begin{aligned}x_1 + 2x_2 + 3x_3 &\leq 5 & (1) \\x_1 + x_2 &\geq 3 & (2) \\x_1, x_2, x_3 &\geq 0 & (3) \\x_1, x_2, x_3 &\in \mathbb{Z}\end{aligned}$$

上下是两个分支
下面这个 $x_2 \leq 0$ ，相当于限制了 $x_2=0$

分支决策：不是原始约束，而是：求解器在搜索过程中 主动做出的选择

黄色表示分支决策



也就是 x_2 等于0和 $x_3 \geq 1$ 不能同时出现

冲突分析：不能同时取 $x_3 \geq 1$ ， $x_2 \leq 0$

这个约束相当于限制了， x_2 等于0和 $x_3 \geq 1$ 不能同时出现

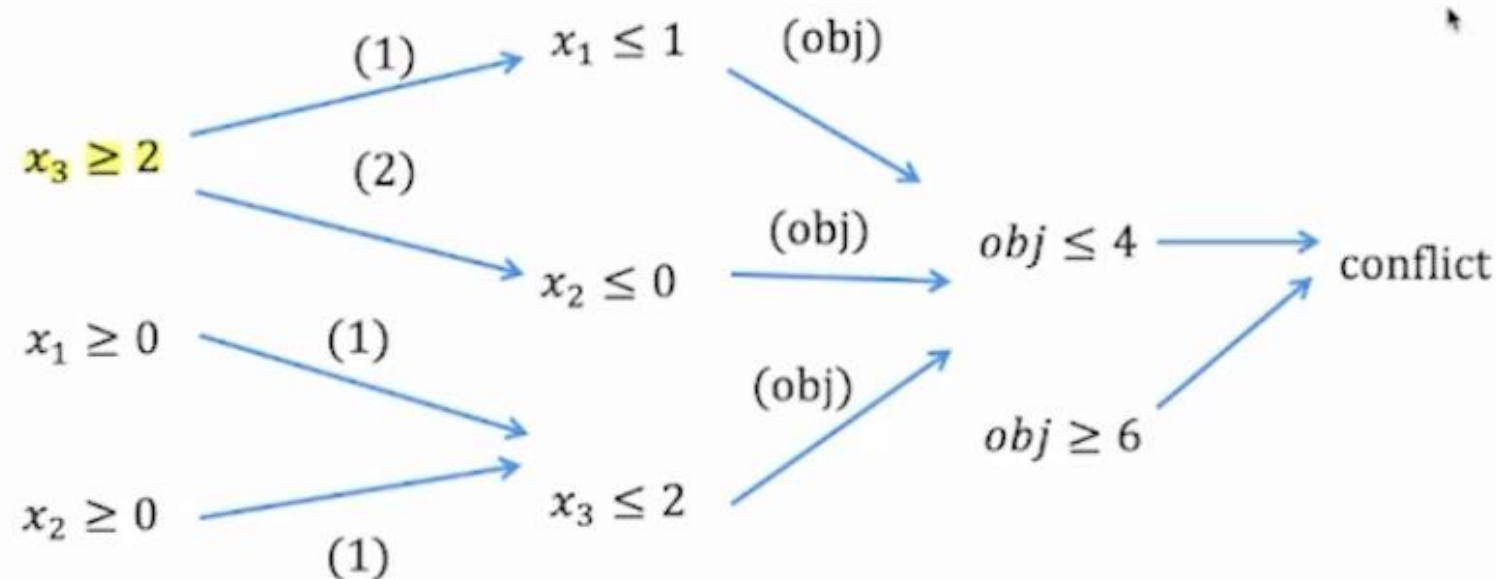
→ 学习到约束： $(1 - x_2) + x_3 \leq 1$

冲突分析

整数线性规划的冲突分析

- 目标值冲突：指当某些变量取某些赋值的时候，目标函数不可能超过已知最优解

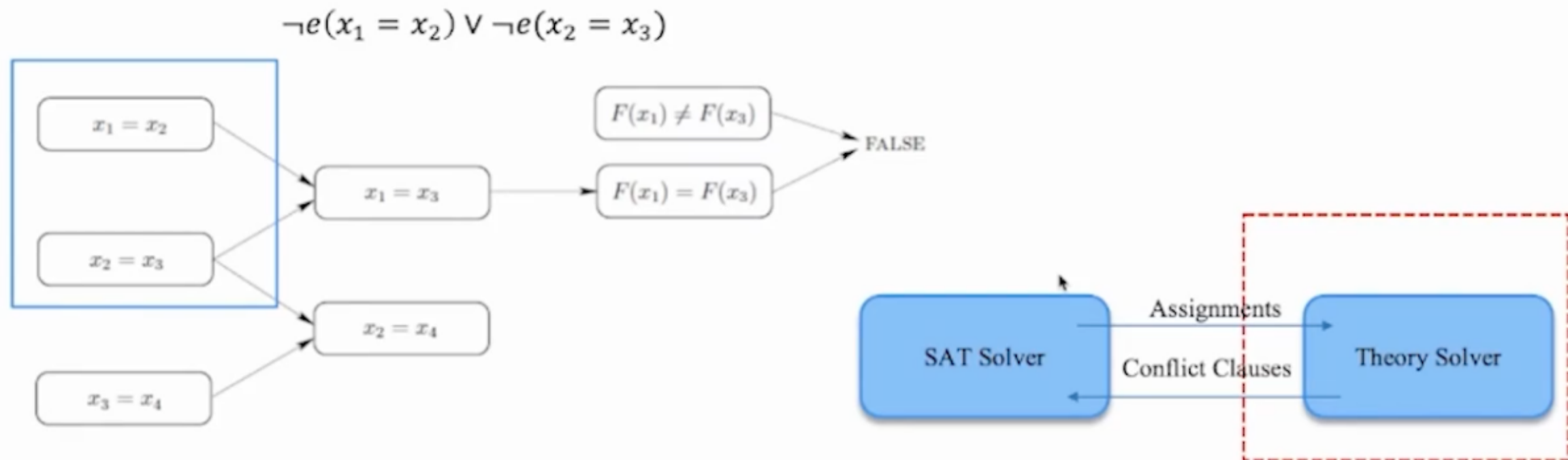
obj: $2x_1 + x_2 + x_3$ (obj)
 $x_1 + 2x_2 + 2x_3 \leq 5$ (1)
 $x_1 + x_2 \geq 3$ (2)
 $x_1, x_2, x_3 \geq 0$ (3)
curObj = 6 (3,0,0) (4)
 $x_1, x_2, x_3 \in \mathbb{Z}$



冲突分析，学习到约束： $x_3 \leq 1$

冲突分析

SMT冲突分析

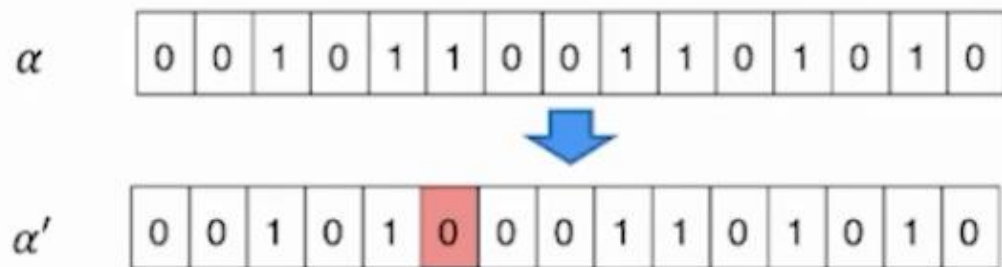


一个等式理论的SMT公式的一段求解过程

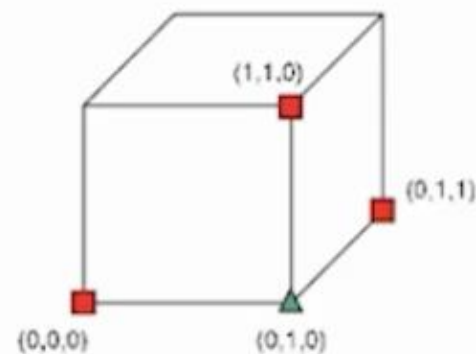
不完备算法---局部搜索

SAT局部搜索

- 从一个完整赋值开始
- 迭代地选择变量进行翻转
- 在搜索空间，从一个点出发，跳到邻居点



两个码字的对应比特取值不同的比特数称为这两个码字的海明距离。



3个变量的SAT公式的搜索空间

局部搜索

- procedure *SLS-Decision*(π)

- input: problem instance $\pi \in \Pi$
- output: solution $s \in S(\pi)$ or $\emptyset$
 $(s) := \text{init}(\pi);$

while not *terminate*(π, s) **do**

$s := \text{step}(\pi, s);$

if $s \in S(\pi)$ then

return s

else

return $\emptyset$

- procedure *SLS-Minimization*(π)

- input: *problem instance* $\pi \in \Pi$
- output: *solution* $s \in S(\pi)$ or $\emptyset$

$(s) := \text{init}(\pi);$

$s^* := s;$

while not *terminate*(π, s) **do**

$s := \text{step}(\pi, s);$

if $f(\pi, s) < f(\pi, s^*)$

$s^* := s;$

if $s^* \in S(\pi)$ then

return s^*

else

return $\emptyset$

不完备算法---局部搜索

局部搜索算法

- 基本操作/算子
- 初始化
- 评分函数
- 搜索启发式

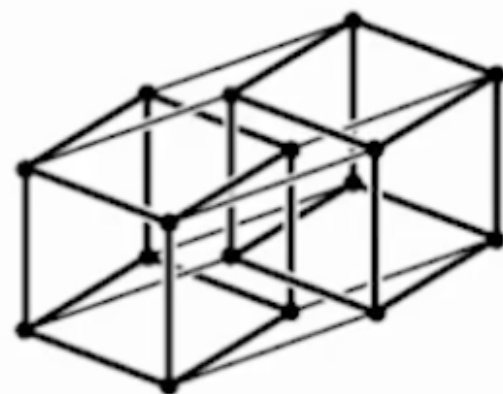
不完备算法---局部搜索

局部搜索算法

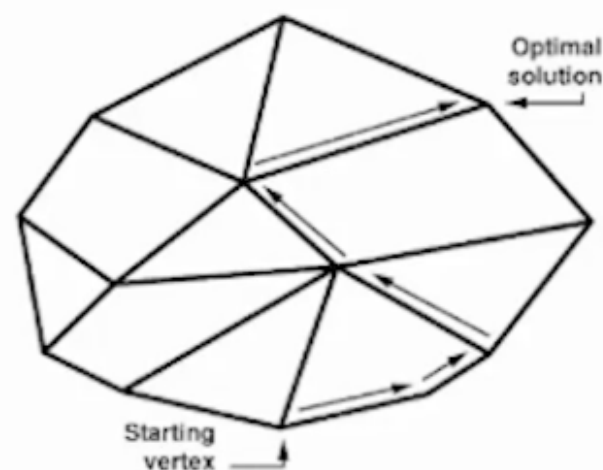
需要定义邻域关系 $\rightarrow$ 一般通过算子定义

- 解空间的结构取决于邻域关系

- 基本操作/算子
- 初始化
- 评分函数
- 搜索启发式



SAT的1海明距离邻域构成n维超立方体，flip操作从一个点跳到另一个邻居点



线性约束定义了多面体可行域

单纯形法从一个可行解开始，移动到另一个（改进的）邻居点

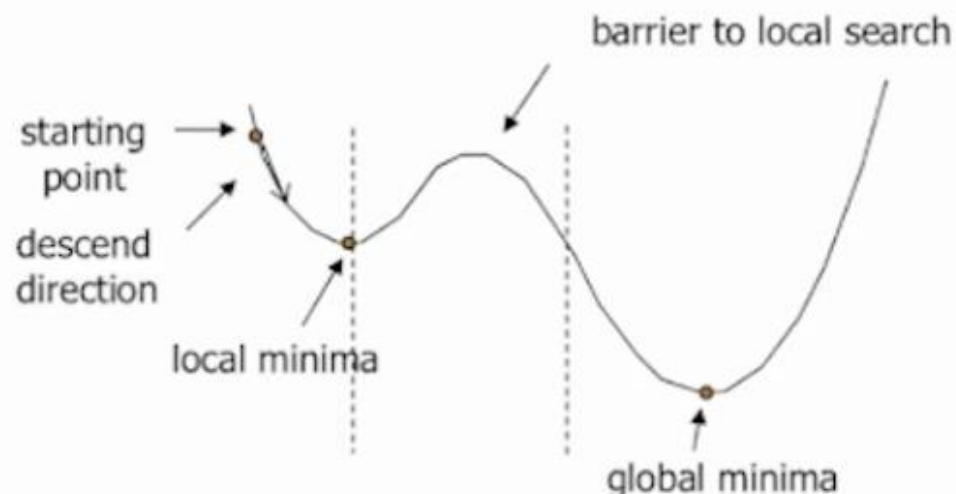
不完备算法---局部搜索

局部搜索算法

- 基本操作/算子
- 初始化
- 评分函数
- 搜索启发式

需要定义邻域关系 → 一般通过算子定义

- 局部最优是相对于一个解空间来说的
- 如何处理局部最优是一个重要问题



不完备算法---局部搜索

局部搜索算法

- 基本操作/算子
- 初始化
- 评分函数
- 搜索启发式

评分函数：用于比较不同操作，从而选择操作

主要考虑问题的目标

- 判定问题：不满足约束的数量
- 约束优化问题：考虑约束和目标
 - 不同的评分函数
 - 综合的评分函数



不完备算法---局部搜索

局部搜索算法

- 基本操作/算子
- 初始化
- 评分函数
- 搜索启发式

评分函数：用于比较不同操作，从而选择操作

主要考虑问题的目标

- 判定问题：不满足约束的数量
- 约束优化问题：考虑约束和目标
 - 不同的评分函数
 - 综合的评分函数



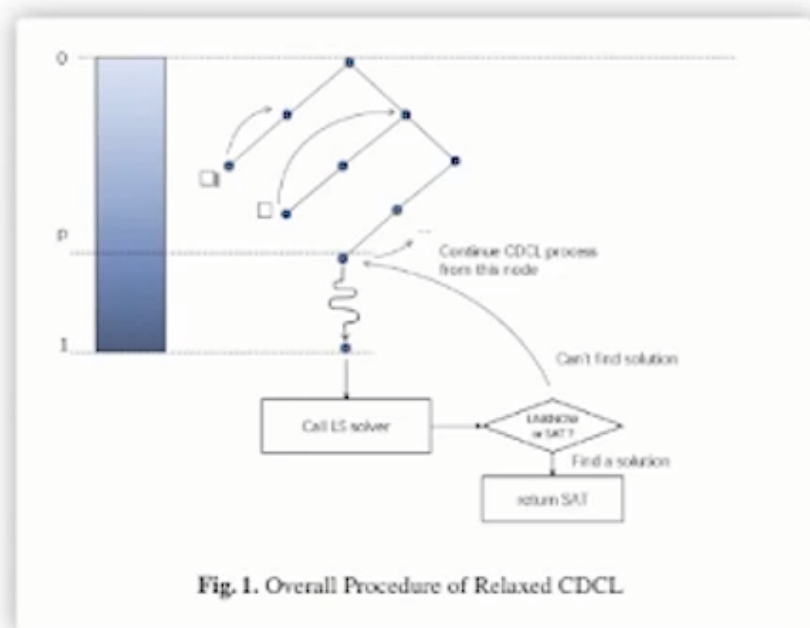
往往也需要考虑算法的行为

- 操作的执行频率
- 访问解的多样性

求解器

- SAT : MiniSAT、Kissat、PRRS (并行)
- SMT: Z3、CVC5
- CSP : Choco、Gecode、Minizinc (建模)
- 数学规划 : Gurobi (商用)、SCIP

混合求解方法



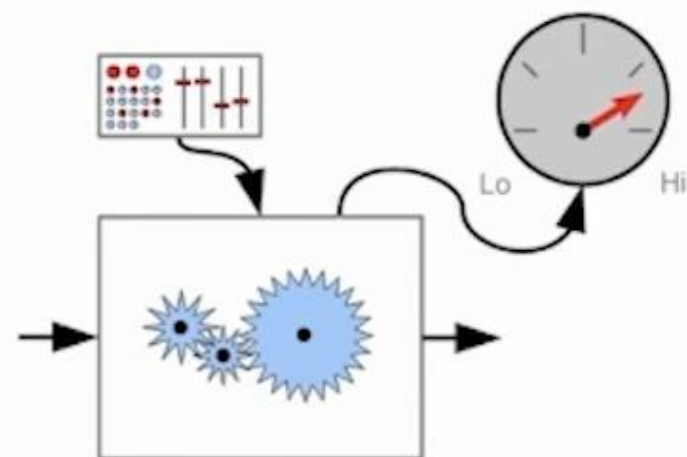
完备算法



局部搜索

自动调参与算法选择

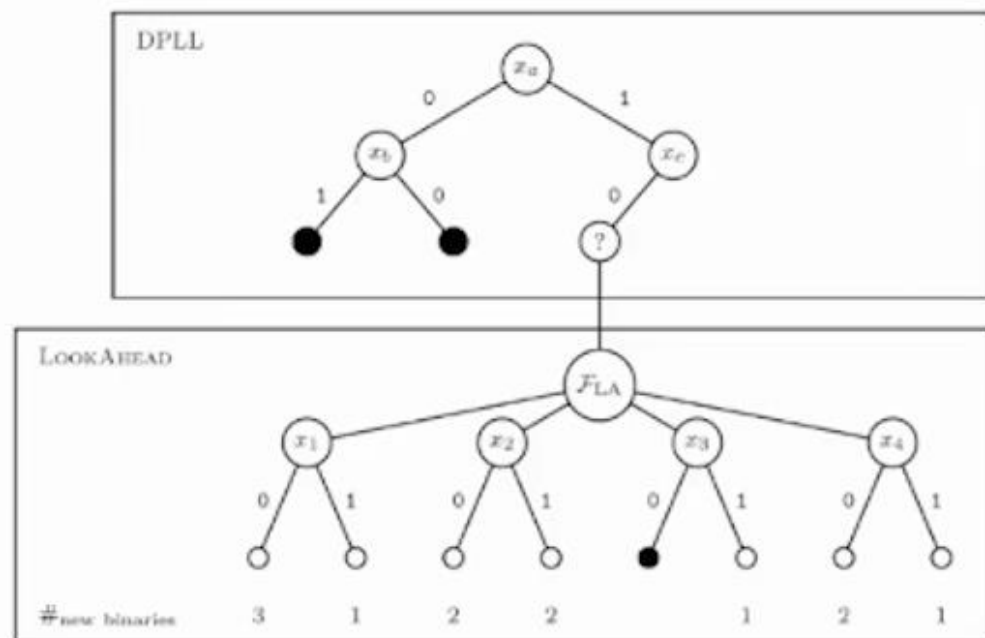
自动调参



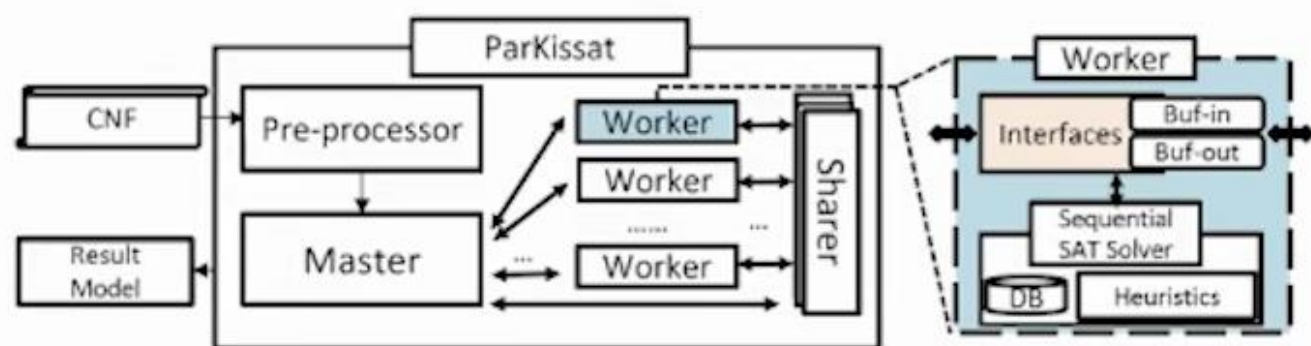
算法选择



并行求解



分治法



多样性方法